

CONTENTS

Práctico 3 (Grafos) — Programación 3	
FING UdelaR · Resolución completa y verificada	
Ejercicio 1 — Modelado de red de antenas	
Ejercicio 2 — Recorridas de grafos	
Ejercicio 3 — DFS iterativo con pila	
Ejercicio 4 (KT 3.2, evaluación) — Detección de ciclos vía DFS	
Ejercicio 5 (KT 3.9, evaluación) — Nodo de corte entre s y t	
Ejercicio 6 (KT 3.10, evaluación) — Cantidad de caminos más cortos	
Ejercicio 7 — Bipartición	
Ejercicio 8 (KT 3.4, evaluación) — Consistencia de juicios sobre mariposas	
Ejercicio 9 — BFS/DFS/SCC sobre grafo dirigido	
Ejercicio 10 — Órdenes topológicos sobre DAG	
Ejercicio 11 (KT 3.3, evaluación) — Orden topológico o ciclo, para dirigido arbitrario	
Ejercicio 12 (KT 3.12, evaluación) — Consistencia de cronología histórica	
Resumen de técnicas usadas en este práctico	

Práctico 3 (Grafos) — Programación 3

FING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

⚠ Nota general: los ejercicios 2(b-e), 9 y 10 piden ejecutar BFS/DFS/orden topológico sobre figuras específicas dibujadas en el enunciado. Acá se explica el **método completo** para resolverlos (que es el contenido evaluable real), pero la traza puntual nodo-por-nodo conviene verificarla con el dibujo original al lado — transcribir de memoria las aristas exactas de un grafo dibujado a mano arriesga errores de transcripción que no aportan nada a entender el método.

Ejercicio 1 — Modelado de red de antenas

(a) Modelado: grafo no dirigido $G=(V,E)$ donde V = las n antenas, y una arista (u,v) representa un cable directo entre las antenas u y v . “Conexión entre todas las antenas” = el grafo debe ser **conexo**. “Menor cantidad de conexiones” = minimizar $|E|$.

(b) ¿ n conexiones es óptimo? No necesariamente óptimo — con n conexiones sobre n nodos conexos, hay al menos un ciclo (un grafo conexo con n nodos y n aristas tiene exactamente 1 ciclo más que un árbol). Se puede quitar una arista del ciclo sin desconectar el grafo, usando $n-1$ conexiones. Por lo tanto n conexiones nunca es óptimo cuando $n \geq 3$ (siempre se puede mejorar en 1).

(c) ¿ $n-2$ conexiones es válida? No es válida en general: un grafo conexo de n nodos necesita **al menos** $n-1$ aristas (ver (d)). Con $n-2$ aristas es imposible que el grafo sea conexo — como mínimo quedan 2 componentes separadas (cada arista puede “unir” a lo sumo 2 componentes previamente separadas en una).

(d) La solución óptima tiene exactamente $n-1$ conexiones. *Cota inferior:* un grafo con n nodos y k componentes conexas tiene al menos $n-k$ aristas (por inducción: agregar un nodo aislado suma 1 al conteo de componentes sin agregar aristas; cada arista agregada reduce la cantidad de componentes en a lo sumo 1). Para que el grafo sea conexo ($k=1$), se necesitan al menos $n-1$ aristas. *Alcanzable:* cualquier árbol de expansión (spanning tree) de n nodos tiene exactamente $n-1$ aristas y es conexo por construcción. Por lo tanto la solución óptima usa exactamente $n-1$ conexiones — y es, precisamente, un árbol de expansión mínimo (MST) sobre las antenas. ■

Ejercicio 2 — Recorridas de grafos

(a) Tres algoritmos distintos para recorrer un grafo:

1. **BFS** (Breadth-First Search): explora por niveles, usando una **cola** (FIFO) para los nodos descubiertos-pero-no-procesados.
2. **DFS** (Depth-First Search): explora en profundidad, usando una **pila** (LIFO) o recursión.
3. **Exploración genérica** (KT 3.2): mantiene un conjunto S de nodos descubiertos sin orden específico — cualquier estructura de datos (ni cola ni pila) que solo garantice “sacar algún elemento” ya alcanza para explorar todo el grafo; BFS y DFS son casos particulares de este esquema general según la estructura elegida.

(b)-(e) Requieren la Figura 2.1 del enunciado. **Método:**

- **BFS desde nodo 8:** nivel 0={8}; nivel 1=vecinos directos de 8; nivel 2=vecinos de esos no visitados aún; etc. El árbol BFS conecta cada nodo con el primer nodo del nivel anterior que lo descubrió.
- **¿Único árbol BFS?** No en general — si un nodo tiene más de un vecino en el nivel anterior, cualquiera de ellos puede ser su padre en el árbol BFS; distintos órdenes de procesamiento de vecinos dan árboles BFS distintos (mismos niveles, pero distintas aristas padre-hijo cuando hay ambigüedad).
- **DFS desde nodo 6:** similar pero explorando cada rama hasta el fondo antes de retroceder (backtrack). Tampoco es único en general, por la misma razón (orden de exploración de vecinos).
- **(d)/(e):** en general, el árbol BFS desde un nodo y el árbol DFS desde otro nodo (o el mismo) **no coinciden**, salvo en casos degenerados (ej. un grafo que ya es un camino simple, donde BFS y DFS producen el mismo árbol). Verificar sobre la figura si alguno de los árboles obtenidos en (b)/(c) resulta ser simultáneamente válido como BFS y como DFS — es infrecuente pero no imposible en grafos con estructura particular (ej. si el grafo es él mismo un camino).

Ejercicio 3 — DFS iterativo con pila

(a) DFS iterativo:

```
DFS-Iterativo(G, s):
    Pila.push(s); visitado[s] = true
    mientras Pila no está vacía:
        u = Pila.pop()
        para cada vecino v de u:
            si v no visitado:
                visitado[v] = true
                padre[v] = u
                Pila.push(v)
```

(b) **¿Depende de la implementación? ¿Está sub-especificado?** Sí, depende de la implementación — específicamente del **orden en que se recorren los vecinos** de cada nodo al apilarlos. El algoritmo tal como está descrito (informalmente, “explorar en profundidad”) no especifica ese orden, por lo que dos implementaciones válidas pueden producir árboles DFS distintos (mismo conjunto de nodos visitados, pero distinta estructura de árbol) — el algoritmo está efectivamente sub-especificado en ese punto, aunque el resultado (“¿es alcanzable v desde s ?”, conjunto de componentes conexas, etc.) es el mismo sea cual sea el orden elegido.

(c) **Modificación para retornar el árbol:** guardar, para cada nodo v , un puntero `padre[v]` que apunta al nodo u desde el cual v fue descubierto (como en el pseudocódigo de (a)). El árbol de recorrida es exactamente el conjunto de aristas $\{(\text{padre}[v], v) : v \neq s\}$.

Ejercicio 4 (KT 3.2, evaluación) — Detección de ciclos vía DFS

(a) **Algoritmo:** ejecutar DFS desde cualquier nodo no visitado (repitiendo para cada componente conexas), manteniendo `padre[v]` y un marcador de “en la pila de recursión actual” (`enPila[v]`). Al explorar la arista (u, v) :

- Si v no visitado: recursión normal, marcar $\text{padre}[v]=u$.
- Si v ya visitado **y** $\text{enPila}[v]=\text{true}$ **y** $v \neq \text{padre}[u]$: se encontró un **ciclo**. Reconstruirlo siguiendo punteros padre desde u hacia atrás hasta llegar a v , y agregando la arista (u, v) al final.
- Si v ya visitado pero $\text{enPila}[v]=\text{false}$ (ya se retiró de la recursión, es de otra rama ya explorada): no hay ciclo por esa arista.

(b) **Corrección.** ($\Leftarrow$) *Si el algoritmo devuelve una secuencia:* por construcción, la secuencia es $v, \text{padre}[u], \text{padre}[\text{padre}[u]], \dots, u, v$ — cada paso consecutivo es una arista real del grafo (por cómo se construyen los punteros padre durante el DFS) y v aparece como ancestro de u en el árbol DFS con la arista de cierre (u, v) existente en G . Esto es exactamente la definición de un ciclo. ($\Rightarrow$) *Si G contiene un ciclo $C=v_1, v_2, \dots, v_k, v_1$:* sea v_i el primer nodo del ciclo visitado por el DFS. Como el ciclo es un subgrafo conexo, el DFS eventualmente visita a todos los demás nodos de C explorando a partir de v_i (directa o indirectamente) **antes de que v_i salga de la pila de recursión** — esto se debe a que en un DFS, un nodo permanece en la pila de recursión mientras alguno de sus descendientes en el árbol siga sin terminar de explorarse, y todos los nodos de C son alcanzables desde v_i dentro de C sin salir del “territorio” ya descubierto. En algún momento, el DFS explora la arista que cierra el ciclo hacia v_i mientras v_i sigue en la pila ($\text{enPila}[v_i]=\text{true}$) — esa es exactamente la condición que dispara la detección. ■

(c) Complejidad $O(m+n)$. El DFS visita cada nodo una vez ($O(n)$ total en operaciones de “marcar visitado”) y examina cada arista una constante cantidad de veces (a lo sumo 2 en un grafo no dirigido, 1 en dirigido) — $O(m)$ total en el recorrido de aristas. Mantener en `Pila[·]` y `padre[·]` es $O(1)$ por nodo/arista. Reconstruir el ciclo al final recorre a lo sumo n punteros padre — $O(n)$. Total: $O(m+n)$. ■

(d) ¿Sin DFS? Sí es posible — BFS también detecta ciclos (una arista hacia un nodo ya visitado que no es el padre directo indica un ciclo, en el caso no dirigido), o mediante Union-Find: procesar las aristas una a una, y si una arista (u,v) conecta dos nodos que ya están en el mismo conjunto (misma componente), esa arista cierra un ciclo. Ambas alternativas también logran $O(m+n)$ (o $O(m \cdot \alpha(n))$ con Union-Find, prácticamente lineal).

Ejercicio 5 (KT 3.9, evaluación) — Nodo de corte entre s y t

(a) Demostración de existencia. Sea L_i el conjunto de nodos a distancia exactamente i de s (capas de BFS desde s), $i=0,1,2,\dots$. Como $d(s,t) > n/2$, es decir $d(s,t) \geq \lfloor n/2 \rfloor + 1$, el nodo t **no** pertenece a ninguna capa $L_0, \dots, L_{\lfloor n/2 \rfloor}$.

Todas las capas $L_1, \dots, L_{\lfloor n/2 \rfloor}$ son no vacías: si alguna capa L_i (con $i \leq \lfloor n/2 \rfloor$) estuviera vacía, entonces ninguna capa posterior podría tener nodos tampoco (en BFS, para alcanzar distancia $i+1$ hace falta pasar por un nodo a distancia i) — eso implicaría $d(s,t) < i \leq \lfloor n/2 \rfloor + 1$, contradiciendo la hipótesis. Por lo tanto las $\lfloor n/2 \rfloor$ capas $L_1, \dots, L_{\lfloor n/2 \rfloor}$ son todas no vacías.

Conteo: estas $\lfloor n/2 \rfloor$ capas contienen en total a lo sumo $n-2$ nodos (el total de nodos es n ; s ocupa 1 nodo en L_0 , y t no está en ninguna de estas capas, así que sobran a lo sumo $n-2$ nodos para repartir entre $L_1, \dots, L_{\lfloor n/2 \rfloor}$).

Si **todas** estas $\lfloor n/2 \rfloor$ capas tuvieran 2 o más nodos, sumarían al menos $2\lfloor n/2 \rfloor \geq n-1$ nodos — más de los $n-2$ disponibles. Contradicción. Por lo tanto **al menos una capa L_i ($1 \leq i \leq \lfloor n/2 \rfloor$) tiene exactamente 1 nodo.**

Ese nodo es un corte: en BFS, toda arista de G conecta nodos de capas iguales o adyacentes (nunca “salta” una capa) — porque si una arista conectara capas que difieren en ≥ 2 , el extremo más lejano habría sido descubierto antes, a menor distancia. Por lo tanto, eliminar **todos** los nodos de L_i elimina toda arista entre $\{L_0, \dots, L_{i-1}\}$ y $\{L_{i+1}, \dots\}$, desconectando s (en L_0) de t (en una capa posterior a L_i). Como L_i tiene un único nodo, ese nodo por sí solo es el corte buscado.

■

(b) Algoritmo: BFS desde s , calculando las capas $L_0, L_1, \dots$; recorrer las capas L_1 hasta $L_{\lfloor n/2 \rfloor}$ y devolver la primera que tenga tamaño exactamente 1 (por (a), existe garantizado).

(c) Complejidad: el BFS es $O(m+n)$; recorrer las capas hasta $\lfloor n/2 \rfloor$ y contar tamaños es $O(n)$. Total $O(m+n)$. ■

Ejercicio 6 (KT 3.10, evaluación) — Cantidad de caminos más cortos

(a) Algoritmo: ejecutar BFS desde u calculando, para cada nodo w , su distancia $d(w)$ y además un contador N_w = cantidad de caminos de largo mínimo desde u hasta w . Inicializar $N_u=1$. Procesando los nodos **en orden creciente de distancia** (orden natural de BFS), para cada $w \neq u$:

$$N_w = \sum_{t: (t,w) \in E, d(t)=d(w)-1} N_t$$

(sumar N_t sobre todos los vecinos t de w que están exactamente una capa antes que w).
Devolver N_v al final.

(b) Corrección de la recurrencia. *Paso base:* $N_u=1$ — el único camino de largo 0 desde u hasta sí mismo es el camino vacío. *Paso inductivo:* todo camino más corto hasta w (de largo $d(w)$) debe llegar a w desde algún vecino t vía la última arista (t,w) . Como quitar la última arista de un camino más corto da un camino más corto al penúltimo nodo (propiedad de subestructura óptima de caminos más cortos), y las aristas solo pueden conectar capas adyacentes (mismo argumento del Ej.5), ese vecino t necesariamente tiene $d(t)=d(w)-1$ — no puede ser $d(t)=d(w)$ (el camino no acortaría) ni $d(t)<d(w)-1$ (la arista saltaría una capa, imposible en BFS). Cada camino más corto a t (hay N_t de ellos, por hipótesis inductiva) extendido con la arista (t,w) da un camino más corto distinto a w , y todo camino más corto a w se obtiene de esta forma para exactamente un t y un camino a t — por lo tanto la suma sobre todos los t válidos cuenta exactamente todos los caminos más cortos a w , sin repetir ni faltar ninguno. ■

(c) Complejidad: BFS estándar es $O(m+n)$; el cálculo de N_w para cada w suma sobre sus vecinos — sumando sobre todos los w , esto es $O(\sum_w \deg(w)) = O(m)$. Total $O(m+n)$. ■

Ejercicio 7 — Bipartición

(a) Ejemplo de la realidad: asignación de tareas a dos turnos de trabajo (mañana/tarde) donde cada arista representa “estas dos personas no pueden estar en el mismo turno” (ej. porque comparten un recurso limitado) — el problema de encontrar una asignación válida a 2 turnos es exactamente determinar si el grafo de conflictos es bipartito.

(b) Ciclo de longitud impar $\Rightarrow$ no bipartito. Por absurdo: supongamos G bipartito (partición $V=X \cup Y$, toda arista cruza entre X e Y) y con un ciclo $v_1, v_2, \dots, v_k, v_1$ de longitud k impar. Como toda arista cruza entre las partes, los nodos del ciclo alternan estrictamente $X, Y, X, Y, \dots$: si $v_1 \in X$, entonces $v_2 \in Y, v_3 \in X, \dots, v_i \in X$ si i es impar, $v_i \in Y$ si i es par. Para que la arista de cierre (v_k, v_1) sea válida (cruce de partes), v_k debe estar en la parte opuesta a v_1 , es decir $v_k \in Y$, lo que requiere k par. Pero asumimos k impar — contradicción. Por lo tanto G no puede ser bipartito si tiene un ciclo de longitud impar. ■

(c) Algoritmo basado en DFS: DFS desde cualquier nodo no visitado (por cada componente), asignando $\text{color}[s]=0$. Al explorar la arista (u,v) : si v no visitado, $\text{color}[v]=1-\text{color}[u]$ (color opuesto) y recursión. Si v ya visitado, verificar que $\text{color}[v] \neq \text{color}[u]$ — si $\text{color}[v]=\text{color}[u]$, el grafo **no** es bipartito (se encontró un ciclo de longitud impar, por (b) contrapositiva). Si se completa el DFS sin conflictos, el grafo es bipartito con partición dada por los colores 0/1.

Ejercicio 8 (KT 3.4, evaluación) — Consistencia de juicios sobre mariposas

Formulación unificada: modelar como grafo $G=(V,E)$ donde V =mariposas, y cada juicio da una arista etiquetada “igual” o “diferente”. Buscamos una 2-coloración (especies A/B) tal que las aristas “igual” conecten nodos del mismo color y las “diferente” conecten nodos de distinto color.

(a) Todos los juicios “iguales”: nunca es inconsistente. Asignar la especie A a **todas** las mariposas satisface trivialmente cualquier cantidad de restricciones “igual” (no hay restricción “diferente” que viole). No hace falta ni un algoritmo no trivial — $O(n)$ para recorrer y asignar.

(b) Todos los juicios “diferentes”: puede ser inconsistente. Este es exactamente el problema de bipartición del Ejercicio 7: construir el grafo con las aristas “diferente” y testear si es bipartito (DFS/BFS de 2-coloreo, Ej.7c). Es inconsistente $\Leftrightarrow$ el grafo tiene un ciclo de longitud impar. $O(m+n)$.

(c) Caso general (algoritmo): BFS/DFS de 2-coloreo generalizado. Empezar en cualquier nodo no coloreado, asignarle color A. Al explorar la arista (u,v) : si v no coloreado, asignar $\text{color}[v] = \text{color}[u]$ si la arista es “igual”, o el color opuesto si es “diferente”; si v ya está coloreado, verificar que su color sea consistente con la restricción de la arista (u,v) — si no lo es, el conjunto de juicios es **inconsistente**. Repetir para cada componente conexas (los nodos sin restricciones entre sí pueden colorearse independientemente).

(d) Corrección: el algoritmo asigna colores respetando exactamente las restricciones de cada arista a medida que las procesa (por construcción); si nunca detecta un conflicto, la coloración final resultante satisface **todas** las restricciones simultáneamente (cada arista fue chequeada exactamente en el momento en que ambos extremos quedaron coloreados). Si el algoritmo detecta un conflicto en alguna arista (u,v) , significa que existe un ciclo en el grafo de restricciones cuya paridad de aristas “diferente” es impar respecto al camino ya recorrido desde la raíz — es decir, dos caminos distintos entre u y v (uno vía el árbol de BFS/DFS, otro vía la arista (u,v) directa) imponen colores contradictorios para el mismo nodo, lo cual es una contradicción lógica real de las restricciones — no un artefacto del algoritmo. Por lo tanto el algoritmo detecta inconsistencia **si y solo si** los datos son realmente inconsistentes. ■

(e) Complejidad: igual que BFS/DFS estándar — cada nodo se colorea una vez $O(n)$, cada arista se examina una vez $O(m)$. Total $O(m+n)$. ■

Ejercicio 9 — BFS/DFS/SCC sobre grafo dirigido

Método (a aplicar sobre la figura del enunciado):

- **(a) BFS desde A:** capas por distancia dirigida desde A, siguiendo únicamente aristas en su sentido original.
- **(b) DFS desde B:** recorrida en profundidad siguiendo aristas dirigidas desde B.
- **(c) Sobre A y B:** A y B están en la misma SCC **si y solo si** B es alcanzable desde A (visto en (a)) y A es alcanzable desde B (visible explorando aristas hacia atrás, o completando (b) si el DFS desde B efectivamente alcanza a A).
- **(d) DFS desde A en G^{rev} :** el grafo con todas las aristas invertidas — los nodos alcanzados son exactamente los que **alcanzan** a A en el grafo original.

- **(e) Todas las SCC de G:** técnica de Kosaraju — (1) hacer DFS en G registrando el orden de **finalización** de cada nodo; (2) procesar los nodos en orden decreciente de finalización, haciendo DFS en G^{rev} desde cada nodo no visitado — cada árbol DFS resultante en este segundo paso es exactamente una SCC completa. Alternativamente, con grafos chicos como el de la figura: para cada nodo, calcular el conjunto de alcanzables (DFS en G) y el conjunto de “alcanzadores” (DFS en G^{rev}) — la intersección de ambos conjuntos es su SCC, y agrupando nodos con la misma SCC se obtienen todas.

Ejercicio 10 — Órdenes topológicos sobre DAG

Método (a aplicar sobre la figura del enunciado):

- **(a)/(b):** los nodos sin aristas entrantes son los que pueden ir primeros en cualquier orden topológico — por eso el algoritmo de Kahn los procesa primero, en cualquier orden entre ellos, y va “liberando” nuevos nodos de grado de entrada 0 a medida que se eliminan sus predecesores.
- **(c)** (en G^{rev}): los nodos sin aristas entrantes en G^{rev} son los nodos sin aristas **salientes** en G — es decir, los que pueden ir **últimos** en un orden topológico.
- **(d) Al menos 4 órdenes distintos:** siempre que en algún paso del algoritmo de Kahn haya ≥ 2 nodos disponibles simultáneamente (grado de entrada 0 a la vez), cualquier orden relativo entre ellos es válido — cada punto de ambigüedad multiplica la cantidad de órdenes topológicos posibles.
- **(e) ¿Son todos los posibles?** Solo si se exploraron *todas* las combinaciones de elección en *cada* punto de ambigüedad del algoritmo de Kahn — en general hay que verificar que no quede ninguna combinación sin considerar.
- **(f) Algoritmo y complejidad $O(m+n)$:** algoritmo de Kahn — calcular el grado de entrada de cada nodo ($O(m)$ recorriendo todas las aristas una vez), inicializar una cola con los nodos de grado 0 ($O(n)$), y repetidamente sacar un nodo de la cola, agregarlo al orden, y decrementar el grado de entrada de sus vecinos salientes, agregando a la cola cualquiera que llegue a 0 (cada arista se examina exactamente una vez en total across todas las iteraciones — $O(m)$ — y cada nodo se encola/desencola una vez — $O(n)$). Total $O(m+n)$.
(Corrección: en cada paso, el nodo elegido no tiene predecesores pendientes, por lo que agregarlo respeta el invariante de que todo lo ya ordenado precede correctamente; el algoritmo termina con los n nodos ordenados si y solo si G es un DAG — si quedan nodos con grado >0 al vaciarse la cola, hay un ciclo.)

Ejercicio 11 (KT 3.3, evaluación) — Orden topológico o ciclo, para dirigido arbitrario

(a) Algoritmo: correr el algoritmo de Kahn del Ejercicio 10(f) tal cual. Si al terminar se ordenaron los n nodos, devolver ese orden topológico. Si la cola se vacía antes de ordenar todos los nodos (quedan nodos con grado de entrada >0 entre sí, formando un subgrafo donde todos tienen al menos un predecesor pendiente dentro de ese mismo subconjunto restante), extraer un ciclo: partiendo de cualquier nodo restante, seguir repetidamente **cualquier** arista entrante desde otro nodo restante (existe siempre, ya que todos tienen grado de entrada >0 dentro del subconjunto restante) hasta repetir un nodo — el segmento entre las dos ocurrencias de ese nodo repetido es un ciclo.

(b) Corrección. *Si termina con orden completo:* por invariante de Kahn (Ej. 10f), es un orden topológico válido, lo cual solo es posible si G es un DAG. *Si queda un subconjunto R de nodos sin ordenar:* cada nodo de R tiene, dentro de R , al menos una arista entrante desde otro nodo de R (si no, tendría grado de entrada 0 respecto al subgrafo restante y habría sido procesado). Siguiendo aristas entrantes repetidamente dentro de un conjunto finito R donde todo nodo tiene al menos un predecesor en R , necesariamente se revisita un nodo — el segmento repetido es, por construcción, una secuencia cerrada de aristas reales de G , es decir un ciclo. ■

(c) Complejidad $O(m+n)$. El algoritmo de Kahn ya es $O(m+n)$ (Ej. 10f). La extracción del ciclo, en el peor caso, recorre a lo sumo $|R| \leq n$ nodos antes de repetir uno — $O(n)$ adicional. Total $O(m+n)$. ■

Ejercicio 12 (KT 3.12, evaluación) — Consistencia de cronología histórica

(a) Modelado y algoritmo. *Representación de entrada:* lista de m datos, cada uno (P_i, P_j, tipo) con $\text{tipo} \in \{\text{"murió-antes"}, \text{"solapado"}\}$.

Modelo: las restricciones de tipo 1 (" P_i murió antes de que P_j naciera") son un **orden estricto** entre personas — se pueden modelar directamente como una arista dirigida $P_i \rightarrow P_j$ en un grafo D . Las de tipo 2 ("solapadas en el tiempo") son distintas: no imponen un orden, sino que **fuerzan** a que ambas personas convivan — ninguna puede haber muerto antes de que la otra naciera, en ningún sentido.

Algoritmo:

1. Construir un grafo no dirigido U con una arista $\{P_i, P_j\}$ por cada dato de tipo “solapado”.
Calcular sus componentes conexas (BFS/DFS, $O(m+n)$) — cada componente es un grupo de personas que deben convivir simultáneamente en algún momento común (“generación”).
2. Construir un grafo dirigido D' sobre las **componentes** (no sobre las personas individuales): por cada dato de tipo “murió-antes” (P_i, P_j) , agregar una arista dirigida de la componente de P_i a la componente de P_j . **Si P_i y P_j quedaron en la misma componente** (es decir, algún dato de “solapado” ya los conecta transitivamente) — es una **contradicción directa**: no pueden haber sido simultáneamente contemporáneos y estar uno muerto antes de que naciera el otro. Devolver “inconsistente”.
3. Ejecutar el algoritmo del Ejercicio 11 sobre D' . Si se detecta un ciclo, los datos son inconsistentes (devolver el ciclo como evidencia). Si se obtiene un orden topológico de las componentes, construir la cronología: para cada componente en el orden topológico, hacerla “nacer” toda junta y “morir” toda junta, en una franja de tiempo posterior a la de todas las componentes anteriores en el orden y anterior a las siguientes (ej. componente k -ésima en el orden: nace en el instante $2k$, muere en el instante $2k+1$ — todos sus miembros con las mismas marcas).

(b) Corrección. Si el algoritmo devuelve una cronología: por construcción, dentro de cada componente todos nacen y mueren juntos, satisfaciendo automáticamente todas las restricciones “solapado” (viven exactamente el mismo intervalo). Entre componentes, el orden topológico de D' garantiza que si hay una arista componente(P_i)→componente(P_j), la componente de P_i aparece antes en el orden, por lo que su intervalo de vida $(2k, 2k+1)$ termina antes de que empiece el de P_j ($2k'+2k'$ con $k'>k$) — satisfaciendo “ P_i murió antes de que P_j naciera”. Si el algoritmo devuelve inconsistente: o bien detectó dos personas forzadas a la vez a ser la misma componente (solapadas) y en orden estricto (murió-antes) — contradicción lógica directa; o bien D' tiene un ciclo de restricciones “murió-antes” entre componentes, lo que significaría que una componente debería morir antes de nacer transitivamente — imposible en cualquier cronología real. En ambos casos, no puede existir una cronología válida. ■

(c) Complejidad $O(m+n)$. Paso 1 (componentes de U): $O(m+n)$. Paso 2 (construir D' , chequear mismas-componentes): $O(m)$. Paso 3 (Kahn/detección de ciclo sobre D'): $O(m+n)$ (D' tiene a lo sumo tantas aristas y nodos como el grafo original). Total $O(m+n)$. ■

Resumen de técnicas usadas en este práctico

Ejercicio	Técnica
1	Cota inferior de aristas para conexidad ($n-1$)
2, 3, 9, 10	Mecánica de BFS/DFS, no-unicidad del árbol de recorrida
4, 11	Ciclos vía DFS (back edges) / vía Kahn (nodos atascados con $\text{grado} > 0$)
5, 6	Argumentos sobre capas de BFS (pigeonhole, subestructura óptima)
7, 8	2-coloreo vía BFS/DFS (bipartición, y su generalización a restricciones igual/diferente)
9	SCC vía Kosaraju (G y G^{rev})
10, 11	Orden topológico vía algoritmo de Kahn (grado de entrada)
12	Combinación: Union-Find/componentes + reducción a orden topológico (Ej. 11) sobre un grafo “contraído”

Para el examen: el patrón más transferible de este práctico es la **reducción a un problema ya resuelto** (Ej. 8 reduce a bipartición del Ej.7; Ej. 12 reduce a componentes conexas + orden topológico del Ej.11) — casi todos los “problemas grandes” de grafos en los exámenes de este curso son variaciones de este patrón: modelar la situación real como grafo, identificar cuál de los algoritmos ya vistos (BFS, DFS, ciclos, bipartición, SCC, topológico) resuelve el problema transformado, y adaptar la demostración de corrección y complejidad al caso concreto.